2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid) | 978-1-7281-9586-5/21/$31.00 ©2021 IEEE | DOI: 10.1109/CCGrid51090.2021.00078

ECO: Edge-Cloud Optimization of 5G applications

Kunal Rao
NEC Laboratories America
Princeton, NJ
kunal@nec-labs.com

Giuseppe Coviello
NEC Laboratories America
Princeton, NJ
giuseppe.coviello@nec-labs.com

Wang-Pin Hsiung
NEC Laboratories America
San Jose, CA
whsiung@nec-labs.com

Srimat Chakradhar
NEC Laboratories America
Princeton, NJ
chak@nec-labs.com

***Abstract*—Centralized cloud computing with 100+ milliseconds network latencies cannot meet the tens of milliseconds to sub-millisecond response times required for emerging 5G applications like autonomous driving, smart manufacturing, tactile internet, and augmented or virtual reality. We describe a new, dynamic runtime that enables such applications to make effective use of a 5G network, computing at the edge of this network, and resources in the centralized cloud, at all times. Our runtime continuously monitors the interaction among the microservices, estimates the data produced and exchanged among the microservices, and uses a novel graph min-cut algorithm to dynamically map the microservices to the edge or the cloud to satisfy application-specific response times. Our runtime also handles temporary network partitions, and maintains data consistency across the distributed fabric by using microservice proxies to reduce WAN bandwidth by an order of magnitude, all in an *application-specific manner* by leveraging knowledge about the application's functions, latency-critical pipelines and intermediate data. We illustrate the use of our runtime by successfully mapping two complex, representative real-world video analytics applications to the AWS/Verizon Wavelength edge-cloud architecture, and improving application response times by 2x when compared with a static edge-cloud implementation.**

***Index Terms*—edge-cloud optimization, microservices, runtime, AWS Wavelength, 5G applications**

I. Introduction

Cloud services are everywhere. From individual users watching over-the-top video content to enterprises deploying software-as-a-service, cloud services are increasingly how the world consumes content and data. Although centralized cloud computing is ubiquitous, and economically efficient, an exponential growth in internet-connected machines and devices is resulting in emerging new applications, services, and workloads for which the centralized cloud quickly becomes computationally inefficient [1]. New, emerging applications like autonomous driving, smart manufacturing, tactile internet, remote surgeries, real-time closed-loop control as in Industry 4.0, augmented or virtual reality require tens of milliseconds to sub-millisecond response times [2]. For these applications, processing all data in the cloud and returning the results to the end user is not an option because it takes too long, uses excessive power, creates privacy and security vulnerabilities, and causes scalability problems.

The new applications demand a different kind of computing fabric, one that is distributed and built to support low-latency and high-bandwidth service delivery capability, which centralized cloud implementations with 100+ milliseconds (ms) network latencies are not well-suited for [3]. For such applications, consideration of a few human-related benchmarks is useful. The time to blink an eye is about 150 ms [4], and a world-class sprinter can react within 120 ms of a starting gun [5]. The human auditory reaction time is about 100 ms, and the typical human visual reaction time is in the range of 10 ms [6]. Also, nerve impulses travel at over 100 meters per second in the human body [7], and the time required to propagate a signal from hand to brain is about 10 ms [8]. If application response times are in this range, then it is possible to interact with distant objects with no perceived difference to interactions with a local object. Even faster, sub-milliseconds response times are required for machine to machine communication as in industry 4.0, where closed-loop real-time control systems automate processes like quality control, high-speed manufacturing, food packaging, construction, and e-health, without the involvement of any human. Clearly, if applications are to approach this level of responsiveness, the network latencies (not including processing) must be much less than 100 ms, which 5G network and edge computing can deliver.

A. Edge-cloud

The speed of light places a fundamental limit on the network latencies, and the farther the distance between the data source and the processing destination, the more time it will take to transmit the data to the destination. So, edge computing places computing resources at the edges of the Internet in close proximity to devices, information sources and end-users, where the content is created and consumed. This, much like a cache on a CPU, increases bandwidth and reduces latency between the end-users or data sources, and data processing. Today, centralized cloud has more than 10,000 data centers [9] scattered across the globe, but within the next five years, driven by a need to get data and applications closer to end-users (both humans and machines), orders of magnitude more heavily scaled-down data centers are expected to sprout up at the edge of the Internet to form the *edge-cloud*. A tiered system, as shown in Fig. 1, with the cloud, and additional, heterogeneous computing and storage resources placed inside or in close proximity to the sensor, is emerging as a computing

978-1-7281-9586-5/21/$31.00 ©2021 IEEE
DOI 10.1109/CCGrid51090.2021.00078

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

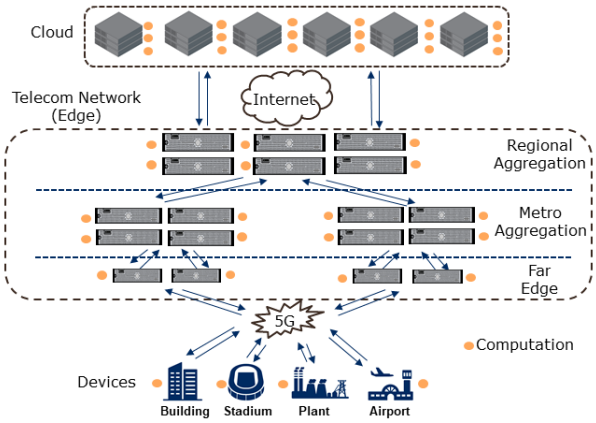


Fig. 1: Tiered, edge-cloud reference architecture

reference architecture for edge-cloud applications. The cloud is expected to be used for a fully centralized application delivery, management and execution of select application functions that may require a global perspective.

1) Advantages: A tiered reference architecture is attractive for several reasons. *First*, computing capability at the edge of the cellular network can enable fundamentally new applications that require high data-rate instantaneous communications, low latency, and massive connectivity, which can be provided by new networks like 5G. *Second*, by extending the cloud paradigm to the heavily scaled-down edge data centers, it is possible for edge providers to quickly develop, install, deliver and manage applications using the same tools and techniques that are used in the cloud. *Third*, cloud can be used for a fully centralized application delivery, and management, in addition to providing computing resources for execution of application functions that require a global perspective.

2) Challenges: Despite its promise, and obvious advantages, the tiered reference architecture also poses several fundamental challenges for applications: *First*, mapping and execution of applications to continuously meet low-latency response times on a dynamic, geo-spatially distributed, and complex edge-cloud infrastructure with heterogeneous resources (different types of networks and computing resources) is a major challenge. *Second*, edge resources (compute, storage and network bandwidth) being severely limited, shared across applications and more expensive than cloud resources, need special strategies to realize economically viable low-latency response applications. *Third*, handling temporary network disruptions in a distributed infrastructure to meet low-latency application response need techniques beyond traditional application-agnostic methods.

3) Our contribution: In this paper, we describe a runtime that enables applications to make effective use of the large-scale distributed platform consisting of a 5G network, computing and storage resources across the cloud, different tiers of edge-cloud, and the devices. Our runtime leverages internal knowledge about the application's microservices, their interconnections, and the critical pipelines of microservices that determine the latency response of the application, to dynamically map the microservices to different tiers of computing and storage resources to achieve application latency goals, maintain data consistency across the distributed storage by using microservice proxies to reduce WAN bandwidth by an order of magnitude, and finally, handle temporary network disconnections, all in an *application-specific manner*. Our runtime continuously monitors data produced and exchanged among the microservices, and uses a novel graph min-cut algorithm to efficiently map the microservices to the edge or the cloud. We illustrate the use of the proposed runtime by successfully mapping two different types of real-world video analytics applications to the AWS/Verizon Wavelength edge-cloud architecture (discussed in Section IV-A). The proposed approach is just as easily applicable to any application that will be packaged in a 5G network slice [10], whose definition is expected to be standardized in the near future.

II. Microservices and Critical Pipelines

Microservices [11] are increasingly becoming popular, especially in cloud services. They are independently deployable with an automated deployment mechanism, they need a bare minimum of management, they can be built in different programming languages and employ different data storage technologies, and each microservice can be independently updated, replaced and scaled. We focus on such microservices based applications, and as part of the specification of the application, we expect the developer to specify the critical pipelines that determine the response latency of the application from measurement to action, and the acceptable response latencies for the critical pipelines. Applications can have a large number of microservices, with complex interactions, and multiple latency-critical pipelines. Please note that individual latencies of the microservices do not have to be specified, only the desired aggregate latency of the critical pipelines. These microservices are packaged as docker images, and they run as docker containers inside pods in a third-party orchestration framework like Kubernetes [12]. We consider two different types of real-world video analytics applications and show the critical pipelines for each of them.

A. Investigation and forensics

This application is required by law enforcement agencies to quickly search past history of suspects or criminals. In this application, days or weeks worth of videos need to be processed within few minutes or hours, so that the required information can be made available as soon as possible, to

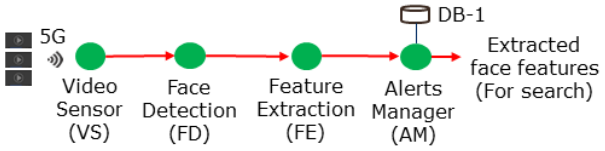


Fig. 2: Investigation and forensics application pipeline

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

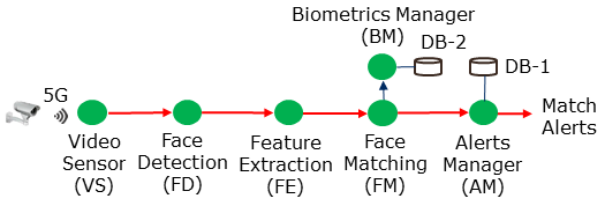


Fig. 3: Real-time monitoring application pipeline

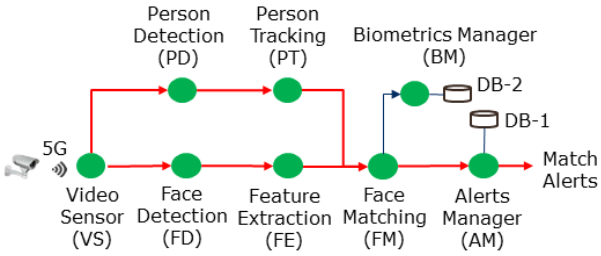


Fig. 4: Application pipeline for variant of real-time monitoring

speed up the investigation process. As a first step towards analyzing these videos, they have to be transferred very quickly for the processing to begin. Next, as the video file processing is on-going and intermediate results are ready, they are continuously made available as soon as possible for further investigative actions. This is a bandwidth-sensitive application; the video file transfer depends on the available upload bandwidth and the continuous reception of results depends on the available download bandwidth. Fig. 2 shows the various microservices involved in this application and the critical pipeline (shown in red). The video files are read by Video Sensor (VS) microservice and frames from these files are made available for further processing. Note that VS can split files and make frames available in batches as well. These frames are then processed by Face Detection (FD) followed by Feature Extraction (FE) microservices to detect faces and extract unique facial feature templates. These extracted features are then made available for search through Alerts Manager (AM) microservice, which stores and retrieves facial features from a persistent store (DB-1). All microservices i.e. VS, FD, FE and AM form the critical pipeline, which determine the response latency of the application, from measurement (i.e. receipt of a video file) to action (i.e. when faces are made available as they are extracted).

B. Real-time monitoring

This is a typical surveillance application where video cameras are used to monitor a particular facility and face-recognition technology is used to flag undesirable individuals like criminals, or to control access and allow only authorized individuals (say, employees) into the facility. This is a latency-sensitive application; the alert is generated as soon as the person of interest is seen (possibly within milliseconds). Fig. 3 shows the entire application as a collection of interconnected microservices and the critical pipeline. The Biometrics Manager (BM) microservice manages names and pictures of individuals to be monitored. Video Sensor (VS) microservice receives the video stream, decodes it and makes it available as individual frames for further processing. Faces in these frames are then detected by the Face Detection (FD) microservice. Once faces are detected, unique facial feature template are extracted by the Feature Extraction (FE) microservice. These feature templates are then matched against the pre-registered individuals' facial feature template by Face Matching (FM) microservice. Based on the match between these facial templates, an alert is generated which is managed by Alerts Manager (AM) microservice. The critical pipeline (shown in red) in this application consists of VS, FD, FE, FM and AM microservices, which determine the response latency of the application, from measurement (i.e. capture of a frame by a video camera) to action (i.e. when an alert is produced).

A common, but more complex variant of the real-time monitoring application is shown in Fig. 4, which involves Person Detection (PD) and Person Tracking (PT) microservices to track individuals when their faces are not always facing the cameras. This application has 2 critical pipelines (shown in red) (a) VS, PD, PT, FM, AM and (b) VS, FD, FE, FM, AM.

Note that all the above applications include microservices that maintain state information in databases. For example, microservices AM and BM each maintain global state in separate databases, and interaction with the databases is not in the critical path of the application. Also, note how the complexity of applications progressively grows from Fig. 2, where there is a single critical path and single DB, to Fig. 3, with single critical path but two DBs and finally Fig. 4, with multiple critical paths and multiple DBs. All these, and even more complex applications are handled by our runtime, which is discussed next in Section III.

III. Runtime for Edge-Cloud

Our runtime consists of two components i.e. Policy Engine (PE) and Scheduler (S), which are themselves implemented as microservices and run as containers within Kubernetes.

1) Policy Engine (PE): This microservice continuously monitors application-level as well as network-level performance metrics, and uses that to determine appropriate partitioning of application in the edge-cloud infrastructure. Each application in our case is a set of microservices, which are chained together in some manner to form a topology or a graph $G = (V, E)$, where the set of vertices $V = (v_1, v_2, ..v_n)$ denotes the microservices and edge $e(v_i, v_j) \in E$ represents the communication between microservice v_i and v_j, where v_i and v_j are neighbors. Each vertex $v \in V$ is assigned with two cost weights $w(v)^{edge}$ and $w(v)^{cloud}$, which are the cost of running the microservice on the edge and cloud respectively. Cost of running microservice v in the edge is given by (1) and cost of running it in the cloud is given by (2).

$$w(v)^{edge} = T_v^{edge} * P_v^{edge} \quad (1)$$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

$$w(v)^{cloud} = T_v^{cloud} * P_v^{cloud} \tag{2}$$

where T_v^{edge} is the execution time of microservice v on the edge, P_v^{edge} is the price (AWS cost) of running the microservice on the edge, T_v^{cloud} is the execution time of the microservice on the cloud and P_v^{cloud} is the price (AWS cost) of running the microservice in the cloud. Note that some microservices cannot be offloaded to the cloud and they have to remain in the edge e.g. microservices that receive inputs from devices in the carrier network or microservices that deliver data to devices in the carrier network. Such microservices are bound to the edge and they only have edge costs.

Each vertex receives one of the two weights depending on where it is scheduled to run i.e. it will get a weight $w(v)^{edge}$ if it is scheduled to run in the edge or $w(v)^{cloud}$ if it is scheduled to run in the cloud. Each edge $e(v_i, v_j) \in E$ represents the communication between v_i and v_j, where v_i is in the edge and v_j is in the cloud (or vice versa), and this edge is assigned a weight given by (3)

$$w(e(v_i, v_j)) = \frac{data_in_{i,j}}{bw_{upload}} + \frac{data_out_{i,j}}{bw_{download}} \tag{3}$$

where $data_in_{i,j}$ is the amount of data transferred (uploaded) from v_i to v_j, $data_out_{i,j}$ is the amount of data received (downloaded) from v_j to v_i, bw_{upload} is the network upload bandwidth and $bw_{download}$ is the network download bandwidth between edge and cloud. Note that when v_i and v_j are both either in the cloud or in the edge, then communication latency between them is considered zero (given by (4) and (5)).

The total latency for the application is the end-to-end time for processing a unit of work, which depends on the time taken by the microservices in the critical pipeline in the application. This total latency is given by (4)

$$\begin{aligned} L_{total} = &\sum_{v \in V} F_v \times T_v^{edge} + \sum_{v \in V} (1 - F_v) \times T_v^{cloud} \\ &+ \sum_{e(v_i, v_j) \in E} F_e \times w(e(v_i, v_j)) \end{aligned} \tag{4}$$

where L_{total} is the sum of the edge latency i.e. processing time taken by microservices on the edge for a unit of work, cloud latency i.e. processing time taken by microservices on the cloud for a unit of work and communication latency i.e. time taken for data transfer between the edge and the cloud. Flags F_v and F_e in (4) are defined as follows:

$$F_v = \begin{cases} 1, & \text{if } v \in V^{edge} \\ 0, & \text{otherwise} \end{cases} \text{ and } F_e = \begin{cases} 1, & \text{if } e \in E_{cut} \\ 0, & \text{if } e \notin E_{cut} \end{cases} \tag{5}$$

where V^{edge} is the set of vertices (microservices) scheduled to run on the edge and E_{cut} is the set of edges $e(v_i, v_j)$ in which v_i and v_j are scheduled on edge and cloud or vice versa.

We formulate the total cost given by (6)

$$\begin{aligned} Cost_{total} = &c_{edge} \times \sum_{v \in V} F_v \times w(v)^{edge} \\ &+ c_{cloud} \times \sum_{v \in V} (1 - F_v) \times w(v)^{cloud} \end{aligned} \tag{6}$$

where the total cost is the sum of the edge computation cost and cloud computation cost, and weight parameters c_{edge} and c_{cloud} can be used to adjust the relative importance between them. The goal of partitioning is to find a cut in the graph $G = (V, E)$ with minimum total cost under given total latency constraint per unit of work. This latency constraint is part of application specification, and PE adheres to this constraint while determining the cut in the graph. This cut separates the graph into two disjoint sets, where one side of the cut is scheduled on the edge while the other side is scheduled on the cloud.

As an example, consider an application with three microservices M_1, M_2 and M_3 chained in a sequential manner as shown in Fig. 5. All three microservices form part of the critical pipeline. Let execution times of M_1, M_2 and M_3 on edge be ${T_{M_1}}^{edge}$, ${T_{M_2}}^{edge}$ and ${T_{M_3}}^{edge}$ respectively, while execution times on cloud be ${T_{M_1}}^{cloud}$, ${T_{M_2}}^{cloud}$ and ${T_{M_3}}^{cloud}$ respectively. Let the cost of running M_1, M_2, and M_3 on edge be $w(M_1)^{edge}$, $w(M_2)^{edge}$ and $w(M_3)^{edge}$ respectively, while the cost of running in the cloud be $w(M_1)^{cloud}$, $w(M_2)^{cloud}$ and $w(M_3)^{cloud}$ respectively. Also, let the communication latency between M_1 and M_2 be $w(e(M_1, M_2))$, while the communication latency between M_2 and M_3 be $w(e(M_2, M_3))$. Based on the partition scheme, if the cut on the graph formed by these microservices is determined to be between M_2 and M_3, such that M_1 and M_2 are scheduled on the cloud and M_3 is schedule on the edge, then value of F_v for M_3 would be 1, while it will be 0 for M_1 and M_2 ((5)). Also, value of F_e for $w(e(M_2, M_3))$ will be 1, while it will be 0 for $w(e(M_1, M_2))$ (again (5)). The total latency given by (4) will then translate to sum of ${T_{M_3}}^{edge}$ (edge latency), ${T_{M_1}}^{cloud}$, ${T_{M_2}}^{cloud}$ (cloud latency) and finally $w(e(M_2, M_3))$ (communication latency) (shown in (7)). And, the total cost given by (6) will translate to sum of $w(M_3)^{edge}$ (edge computation cost), $w(M_1)^{cloud}$ and $w(M_2)^{cloud}$ (cloud computation cost) (shown in (8)).

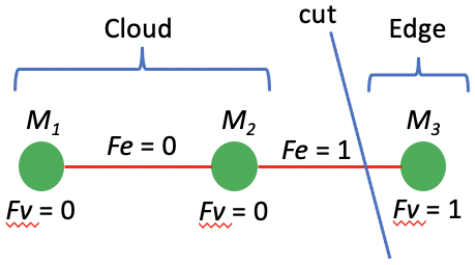


Fig. 5: Microservices partitioning example

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

$$L_{M_1,M_2,M_3} = {T_{M_3}}^{edge} + {T_{M_1}}^{cloud} + {T_{M_2}}^{cloud} + w(e(M_2, M_3)) \quad (7)$$

$$C_{M_1,M_2,M_3} = w(M_3)^{edge} + w(M_1)^{cloud} + w(M_2)^{cloud} \quad (8)$$

In scenarios where there are multiple layers of computing infrastructure available, such that the cost reduces as you go to upper layers at the expense of increased latency, the same method can be applied iteratively across layers to identify the appropriate allocation and scheduling of microservices. For example, consider three computing layers A, B and C, with A being at the top, B in the middle and C at the bottom. Cost of computing is lower as you go up from C to A, while the latency is higher as you go up from C to A. In this scenario, our partitioning scheme will first consider C as the edge and B as the cloud for equations (1), (2) and (3). Once this partition is determined, certain microservices will be scheduled to run on C (edge), while others will be scheduled to run on B (cloud). Then, for the microservices that are scheduled to run on B, and only for these microservices, our partitioning scheme will be applied again. This time B is considered as the edge and A as the cloud for equations (1), (2) and (3). The set of microservices will now be split to run between B (edge) and A (cloud). This way, the various microservices will be allocated and scheduled to run on layers A, B and C. This iterative process can be extended to any number of computing layers and an effective partitioning of microservices can be realized across the various computing layers. By going in this bottom up fashion, we ensure that latency constraint is met at each iteration and cost goes down in successive iterations.

PE also transparently introduces proxy microservices as needed within the application pipeline. For example, in real-time monitoring application, AM is in the critical pipeline but it also interacts with a persistent storage (database) which stores alerts globally. Having AM and the storage in the cloud, with a global perspective, is ideal but it affects the latency of the critical pipeline. To alleviate this problem, PE introduces Alerts-Manager at Edge (AM-E) microservice, which is a proxy for alerts-manager microservice in the cloud. AM-E receives alerts from application microservices running locally in the edge and it publishes the alerts over a ZeroMQ channel for other applications to consume. These alerts are maintained in a temporary buffer and synchronized in the background with AM microservice's persistent storage in the cloud. Addition of AM-E proxy does not add any overhead, rather it expedites the critical pipeline since delivery of alerts happens directly from the edge and synchronization with AM happens in the background without affecting the critical pipeline.

2) Scheduler (S): Scheduler manages the actual scheduling of application components between edge and cloud. These decisions are made (a) statically i.e. at the start of the application and (b) dynamically i.e. while the application is running.

Before the execution of the application starts, based on application and network parameters i.e. the current network condition, and apriori knowledge about execution times and communication across various microservices of the application, the partitioning determined by PE is used by S to decide where to schedule the various microservices. This apriori knowledge, used at the start, is obtained by profiling the application and identifying the execution times of microservices and communication patterns between them.

Algorithm 1 Application scheduling

```
1: while true do
2:   for app ∈ apps do
3:     if !isAppScheduled(app) OR
4:     conditionsChanged(app, a_p, n_p) then
5:         partition ← getPartition(app, a_p, n_p)
6:         scheduleApp(app, partition)
7:     end if
8:   end for
9:   sleep(interval)
10: end while
```

After the application starts running, scheduler S continuously receives application-level and network-level performance data from PE. This is used to periodically check if the previously selected partition is still good or needs to be adjusted dynamically based on the changing environmental (application-level and/or network-level) conditions. If the application performance and/or network performance degrades, resulting in latency constraint for the application not being met by the current partition, then the conditions are considered to have changed and new partitioning is determined to adjust to the changed conditions.

Algorithm 1 shows the scheduling algorithm used by S to schedule microservices in the application. The *getPartition(app, a_p, n_p)* function uses the formulation and graph min-cut technique described in Section III-1 to obtain the partition and *scheduleApp(app, partition)* function uses underlying Kubernetes orchestration framework to schedule and run the application. Note that the individual microservices (and their state) are already pre-loaded at different tiers and the orchestration framework can start or stop running any microservice on any tier, as soon as the signal is received from S.

IV. Experimental results

A. Test bed for 5G applications in edge-cloud

We use AWS Wavelength [13] to explore edge-cloud solutions for 5G applications, in which AWS compute and storage is placed directly within the communication provider's network. This removes the latency that results from multiple hops between regional aggregation sites and across the Internet, and enables new, low-latency applications. Table I shows the configuration for various instance types available in AWS Wavelength. A typical AWS Wavelength infrastructure is shown in Fig. 7, where the Amazon Virtual Private Cloud (VPC) is extended to include the Wavelength Zone and supported EC2 instances are spawned in the Wavelength zone to

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

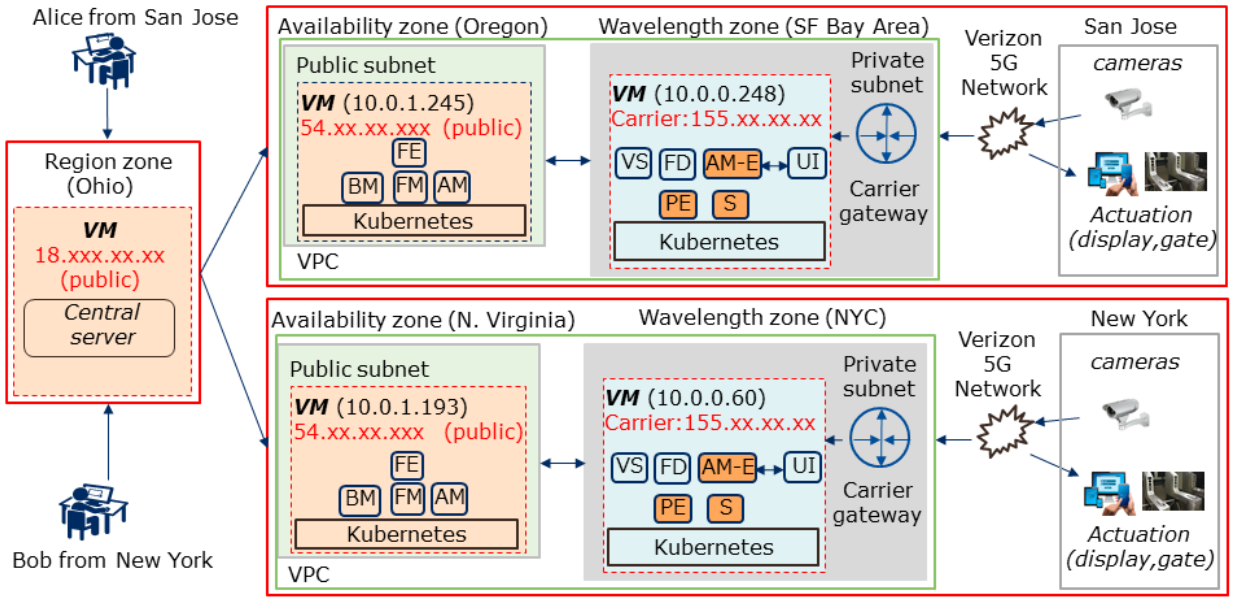


Fig. 6: Test bed for 5G applications in edge-cloud

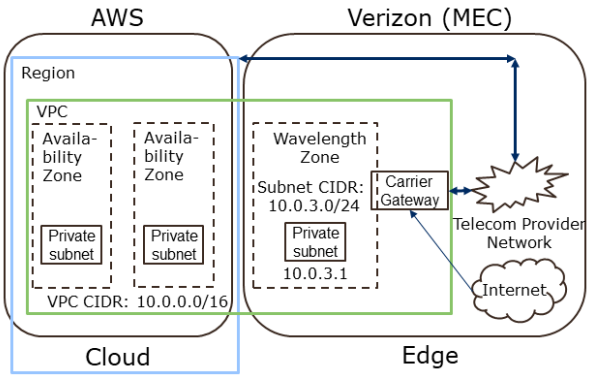


Fig. 7: AWS Wavelength Infrastructure

handle latency-sensitive application components. 5G devices in the carrier network connect to the Wavelength Zone through the carrier gateway and network traffic from these devices is directly routed to the VMs in Wavelength zone without leaving the carrier network.

Our test bed for 5G applications in edge-cloud infrastructure is shown in Fig. 6 and consists of two separate MicroK8s Kubernetes clusters in the Availability (cloud) and Wavelength (edge) Zone. Video feed from cameras is routed into VM in Wavelength zone over Verizon 5G network, and the VMs in Wavelength zones and Availability zones are configured to be in the same VPC. For all our experiments, we used Verizon 5G MIFI 2100 hotspot and t3.xlarge instance type in Amazon AWS infrastructure for Wavelength and Availability Zone.

TABLE I: AWS Wavelength instance types

Instance Types	vCPUs	Memory (GiB)	GPU	Network Perf. (Gigabit)	Price (per hour)
t3.medium	2	4	no	upto 5	$0.056
t3.xlarge	4	16	no	upto 5	$0.224
r5.2xlarge	8	64	no	upto 10	$0.68
g4dn.2xlarge	8	32	yes	upto 25	$1.317

B. 5G and WAN network latencies

In this section we present some raw network performance numbers. Although Verizon claims to have 5G speed in San Francisco bay area, the location where we measured, did not get the full expected 5G speed and therefore we measured at two different locations within the SF Bay area. We report the numbers we observed using this 5G hotspot at these two locations, namely location-1 (GPS coordinates: 37.351368, -121.994782) and location-2 (GPS coordinates: 37.349002, -121.993945). Table II summarizes the latency, upload bandwidth and download bandwidth that we observed between (a) Device in Wavelength zone to VM in Wavelength zone (b) Device in non-wavelength zone (since user can connect from anywhere) to VM in Availability zone and (c) VM in Availability zone to VM in Wavelength zone. We used ping to measure the latency and iperf3 [14] to measure the upload and download bandwidth. We repeat the experiments multiple times and report the minimum, average, maximum and the standard deviation values for all measurements.

From Table II, we can see that the latency between device to VM in Wavelength zone is much lower than that between device to VM in Availability zone, indicating faster turn-around time for responses from Wavelength zone compared to

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Network performance at location-1 in SF Bay area

Metrics	Min.	Avg.	Max.	Std. dev.
Latency between (ms)				
(a) Device and Wavelength	22	39.7	81	9.94
(b) Device and Availability	66.05	78.88	243.5	20.89
(c) Wavelength and Availability	11.4	25.8	34	3.66
Upload Bandwidth (Mbits/s)				
(a) Device to Wavelength	22.2	28.43	37	3.17
(b) Device to Availability	0.8	2.62	7	1.41
(c) Wavelength to Availability	21	35.47	71	10.09
Download Bandwidth (Mbits/s)				
(a) From Wavelength to Device	59	151.3	183	17.05
(b) From Availability to Device	9	31.74	56	12.34
(c) From Availability to Wavelength	20.6	38.31	74	10.54

TABLE III: Network performance at location-2 in SF Bay area

Metrics	Min.	Avg.	Max.	Std. dev.
Latency (milli-seconds)				
(a) Device to Wavelength	24.6	30.9	50.9	4.5
Upload Bandwidth (Mbits/s)				
(a) Device to Wavelength	40	42.8	46	2.4
Download Bandwidth (Mbits/s)				
(a) Wavelength to Device	255	292	318	18.25

those from Availability zone (where the standard deviation is also quite high). Also, the latency between VM in Availability zone and the VM in Wavelength zone is very low, indicating that we can offload processing to the availability zone and get back response from there to Wavelength zone quickly. With respect to upload bandwidth, we see that from device to VM in Wavelength zone, the upload bandwidth is quite high compared to the one to VM in availability zone. This shows that high-resolution video can be streamed to VM in Wavelength zone at high FPS, whereas it will be slow to stream to the VM in Availability zone. Download bandwidth from VM in Wavelength zone to device is quite high while from VM in Availability zone to device is relatively low. This indicates that it will not be possible to receive high network traffic from VM in availability zone to device, but it is possible to obtain from VM in Wavelength zone. If we see the upload and download bandwidth between VM in availability zone and VM in Wavelength zone, we can see that it is high enough to offload processing to availability zone VM and receive back responses on wavelength zone VM.

Table III summarizes the latency, upload and download bandwidth measured between device in Wavelength zone to VM in Wavelength zone at location-2. We are not reporting (b) and (c) from Table II in this table, since they are not directly affected by the 5G hotspot network performance at different locations. We notice that the latency is slightly lower, upload bandwidth is slightly higher, and download bandwidth is significantly higher at location-2 than location-1.

C. Application-processing latencies

All measurements in Section IV-B were application-independent i.e. they were just related to network performance.

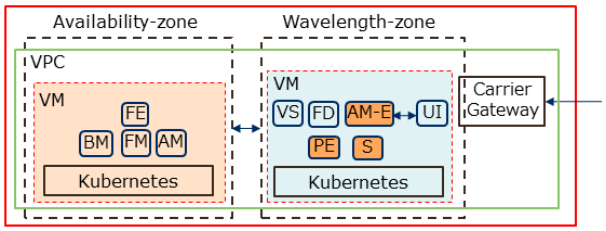


Fig. 8: Hybrid deployment determined by our runtime

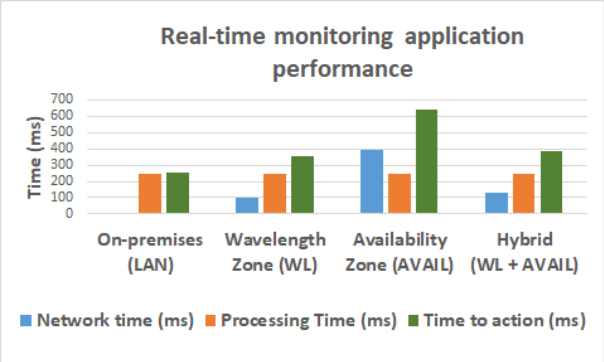


Fig. 9: Real-time monitoring application performance

In this section we report some application-specific performance numbers. Again, we run the application's microservices within two separate Kubernetes clusters in the availability and wavelength zone as shown in our test bed in Fig. 6. Our runtime manages deployment and co-ordination between these separate clusters.

1) Real-time monitoring: Fig. 9 shows the performance for real-time monitoring application for different deployment scenarios. Here we took one full HD frame and measured the time it takes to process this frame i.e. detect and match faces in the frame. This is the reported processing time. Note that we used similar hardware for different deployment scenarios and therefore processing time is the same across various scenarios. Next, we measured the network time i.e. the time spent in sending full HD frame over the network for processing and then receiving back the alert. We measured this time when processing was (a) On premises i.e. in local network (LAN) (b) On VM in Wavelength zone (edge-only) (c) On VM in Availability zone (cloud-only) and (d) Hybrid deployment (shown in Fig. 8 determined by our runtime based on Section III-1). As expected, the total time to action for (a) is the least since everything is in the internal local network. Next is the time taken for (b), since Wavelength zone provides high bandwidth and low latency communication. This is followed by (d) which leverages the high bandwidth and low latency communication provided by Wavelength zone, and between Wavelength and Availability zone. This deployment reduces total cost (details provided in Section IV-D3) by offloading some processing to availability zone at the expense of slightly increased total time to action, but still being within the latency

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Time (in seconds) to upload video files

Deployment scenario	Time to upload video-1 (s)	Time to upload video-2 (s)	Time to upload video-3 (s)
(a) On Wavelength Zone	553	33	61
(b) On Availability Zone	3925	254	387

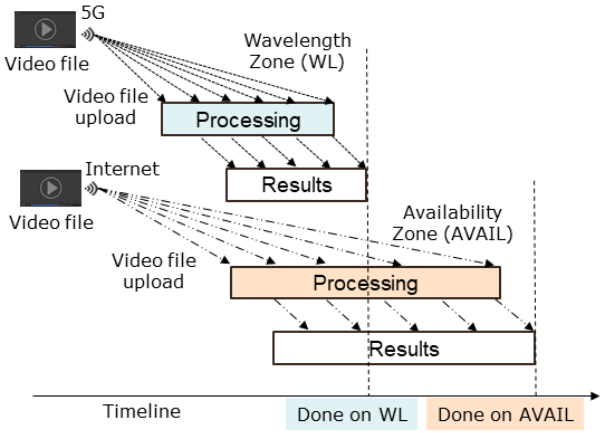


Fig. 10: Investigation and forensics application processing on wavelength and availability zone

constraint. Finally the time taken by (c) is the highest since the network performance for Availability zone is poor compared to Wavelength zone.

2) Investigation and forensics: As mentioned in Section II-A, processing of archived video files for investigation and forensics purpose, depends on how fast the video files can be uploaded to the computing fabric, where actual processing would happen. Table IV shows the time it takes to upload three different video files to VM in Wavelength zone and VM in Availability zone. We see that the time to upload to Wavelength zone is an order of magnitude faster than uploading to the availability zone. As the upload continues, the processing starts immediately and intermediate results are received back to the devices in the carrier network. Fig. 10 shows the overall processing of the video files in two different deployment scenarios i.e. on wavelength zone and availability zone. As shown in the figure, the uploading of file to wavelength zone is faster, compared to uploading on availability zone. By increasing the parallelism to adjust processing rate according to the upload rate, processing the video file is done much sooner (around 7x faster) on wavelength zone than on availability zone.

D. Impact of runtime optimization

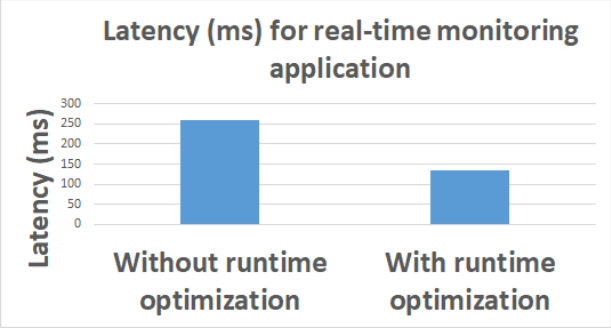
In this section, we evaluate the impact of runtime optimization on application-processing latencies and cost. Particularly, we evaluate the scenario for static as well as dynamic mapping of microservices. We note that sometimes it is possible that our runtime cannot satisfy the response latency constraint, and in such cases this is notified to the developer. However, in our experiments such a scenario did not arise.

1) Impact on static mapping: Fig. 11a shows the network latency for static mapping of microservices of real-time monitoring application with and without runtime optimization. Without using our runtime, one way to deploy the application is to obtain the camera feed directly from carrier network into the Wavelength zone (leveraging low latency and high bandwidth offered by 5G) and then from there stream it into the cloud, where entire application processing happens (reducing cost of processing since VMs in cloud are cheaper) and then get back results through Wavelength zone VM back to the devices. For such a deployment, the overall network time is 260 milliseconds vs 134 milliseconds when hybrid deployment (see Fig. 8) determined by our runtime optimization is used. Our runtime optimization appropriately maps application's microservices on the edge and the cloud to keep the overall latency low.

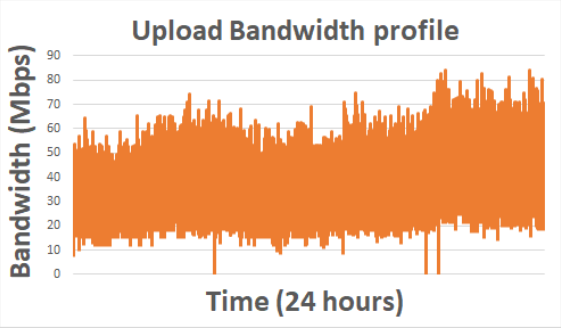
2) Impact on dynamic mapping: Along with the initial static mapping, as discussed in Section III-2 our runtime continuously monitors the application-level and network-level parameters and dynamically updates the mapping of microservices, if conditions change. To see this in action, we deployed the real-time monitoring application with latency constraint of 250 milliseconds, and observed the application and network behavior over a period of 24 hours, with and without the dynamic mapping initiated by our runtime. As shown in Fig. 11b we observe that the upload bandwidth between wavelength and availability zone drops to almost zero for 3 occasions in the 24 hour period. Fig. 11c shows that for each of these three occasions, the latency suddenly spiked when dynamic mapping was turned OFF, whereas as shown in Fig. 11d, when dynamic mapping was turned ON, the spike was handled seamlessly by our runtime by moving FE and FM microservices temporarily from availability zone to the wavelength zone, thereby keeping the end-to-end application latency below 250 milliseconds (specified constraint), despite network disruption.

3) Impact on cost: Based on Amazon pricing [15], Fig. 12 shows the impact our runtime optimization has on the cost of deployment of 100-cameras. For a static mapping as discussed in Section IV-D1 without using our runtime, the number of VMs in Wavelength zone to stream the video to availability zone would be around 15 (required bitrate per camera to stream full HD video at 30 FPS is about 4 to 5 Mbits/s and total upload bandwidth between wavelength zone and availability zone is around 35 MBits/s, so 1 VM can support 7 cameras; to support 100 cameras around 15 VMs would be required) and number of VMs for processing in availability zone would be 50 (1 VM can support 2 cameras; so for 100 cameras 50 VMs would be needed), which brings the total cost per month to around $9842.7, whereas the total cost per month for the hybrid deployment (see Fig. 8) determined by our runtime optimization would be around $8253 per month. Our runtime optimization thus improves network latency by around 2x, while bringing the cost down by around 16% for a 100-camera deployment.

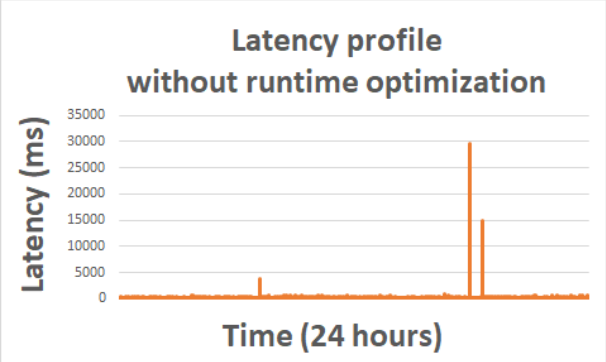
Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.



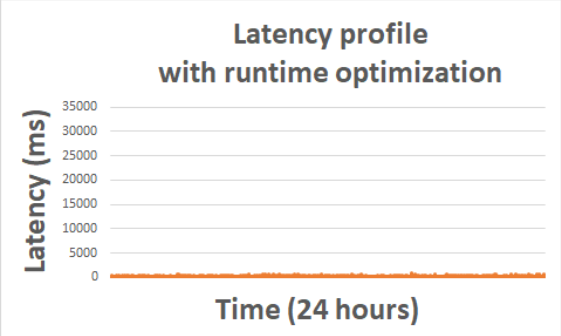
(a) Static mapping



(b) Bandwidth profile



(c) Dynamic mapping OFF



(d) Dynamic mapping ON

Fig. 11: Impact on latency

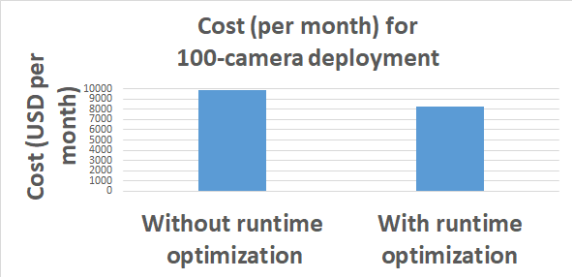


Fig. 12: Impact on cost

V. Related Work

Microservices-based architecture has become quite popular in the past few years [16] [11] and is inspired by the concepts from service oriented architecture [17]. In this architectural style, monolithic applications are broken down into individual microservices, which are developed and deployed independently with lightweight interfaces between them for communication. This naturally results in loosely-coupled set of application microservices, which can be distributed across different computing fabric and can be scaled on-demand, making it a promising path towards implementing distributed IoT services [18]. Teemu et al. [19] showed such a path in designing and implementing distributed IoT applications using microservices. With typical IoT infrastructure, computing capability close to the source of data (devices) is fast but limited, while as you go further away e.g. into MEC [20] and then into the cloud, the computing capability goes up, at the cost of higher latency [21] [22].

With the flexibility provided by the microservices architecture, it is possible to map various components onto the device, MEC or the cloud. However, identifying which parts of the application needs to run where is a challenge [23] and several partitioning and offloading techniques have been discussed by Jianyu Wang et al. [24] and Varshney et al. [25] in their survey paper. Techniques to optimally fuse and place functions in edge or cloud in serverless platforms have been proposed by Tarek Elgamel et al. [26]. Ren et al. [27] formulated the problem of splitting task between edge and cloud as purely latency minimization problem with constraints on communication and computation resources. Anirban Das et al. [28] present models to predict latencies and cost of running tasks in edge or cloud and use that to allocate task appropriately based on provided latency or cost constraints. However, they operate at a single serverless function level rather than looking at a pipeline of microservices to process an input. Techniques to offload some tasks of application from mobile devices to the cloud have been discussed in MAUI [29], CloneCloud [30], and by Huaming Wu et al. [31]. They aim to minimize energy consumption at mobile devices while keeping overall application latency low. They do not take into account the monetary cost of execution of

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

tasks locally vs remotely. Moreover, the granularity at which tasks are offloaded is too fine e.g. at individual function-level. Game theory based approaches have also been proposed for optimizing deployments in edge-cloud infrastructure [32] [33] [34]. Fard et al. [35] propose multi-objective scheduling of scientific workflows on heterogeneous computing environments, but their work focuses mainly on static scheduling and does not consider dynamic scheduling.

With the advent of edge computing using 5G [36] [37], new applications are emerging and currently there is no standard way of programming and efficiently managing applications to take full advantage of this new vertically distributed, hierarchical, heterogeneous infrastructure with 5G connection at the edge. Maheswaran et al. [38] proposed a programming language with custom compiler to program applications for edge-cloud. This however, is quite restrictive in terms of deployment of generic microservices. Sumit et al. [39] propose model to provision resources in edge and cloud, but they consider connectivity between two edge-clouds, which does not exist in our setting, where 5G coverage is limited to certain area and only to certain Wavelength zones (direct low-latency connection between wavelength zones isn't currently available). Unlike any of existing work, we propose a runtime system, which enables applications to make effective use of 5G network, computing at the edge of 5G network, and computing in the cloud, while keeping the overall monetary cost of deployment and end-to-end application latency low.

VI. Conclusion

Demand for edge computing is rising because centralized cloud computing with 100+ milliseconds network latencies cannot meet the tens of milliseconds to sub-millisecond response times required for emerging 5G applications like autonomous driving, smart manufacturing, tactile internet, and augmented/virtual reality. In this paper, we present a new runtime that enables applications to make effective use of a 5G network, computing resources at the edge of this network, and additional computing resources in the centralized cloud. Our dynamic, runtime optimizations leverages application-specific knowledge to improve application response latencies by 2x when compared with a static, manually optimized edge-cloud implementation on AWS Wavelength. Although we used carrier-supported 5G edge-cloud, our approach is equally applicable to local 5G implementations. As 5G networks continue to improve, and the demand for edge computing continues unabated, we expect the costs of edge-cloud to drop by an order of magnitude. This will usher in a new era where edge-cloud will be just as ubiquitous as cloud computing is today.

References

[1] S. Vidas, R. Lakemond, S. Denman, C. Fookes, S. Sridharan, and T. Wark, "A mask-based approach for the geometric calibration of thermal-infrared cameras," *IEEE Transactions on Instrumentation and Measurement*, vol. 61, pp. 1625–1635, 06 2012.

[2] D. Kim, "5G Economics – The Tactile Internet," last accessed 25 Novemmber 2020. [Online]. Available: https://techneconomyblog.com/2017/01/22/5g-economics-the-tactile-internet-chapter-2/

[3] N.-N. Dao, W. Na, and S. Cho, "Mobile cloudization storytelling: Current issues from optimization perspective," *IEEE Internet Computing*, vol. PP, pp. 1–1, 01 2020.

[4] "Blink and you miss it!" last accessed 25 November 2020. [Online]. Available: https://www.ucl.ac.uk/news/2005/jul/blink-and-you-miss-it

[5] E. Tønnessen, T. Haugen, and S. Shalfawi, "Reaction time aspects of elite sprinters in athletics world championships." *Journal of strength and conditioning research / National Strength Conditioning Association*, vol. 27, p. 885–892, 04 2013.

[6] T. Ghuntla, P. Gokhale, H. Mehta, and C. Shah, "A comparison and importance of auditory and visual reaction time in basketball players," *Saudi Journal of Sports Medicine*, vol. 14, p. 35, 01 2014.

[7] A. Castelfranco and D. Hartline, "Evolution of rapid nerve conduction," *Brain Research*, vol. 1641, 02 2016.

[8] B. Daley, "It feels instantaneous, but how long does it really take to think a thought?" 2015, last accessed 25 November 2020. [Online]. Available: https://theconversation.com/it-feels-instantaneous-but-how-long-does-it-really-take-to-think-a-thought-42392

[9] A. Loten, "Data-center market is booming amid shift to cloud," 2019, last accessed 25 November 2020. [Online]. Available: https://www.wsj.com/articles/data-center-market-is-booming-amid-shift-to-cloud-11566252481

[10] A. A. Barakabitze, A. Ahmad, R. Mijumbi, and A. Hines, "5G network slicing using SDN and NFV: A survey of taxonomy, architectures and future challenges," *Computer Networks (Elsevier)*, vol. 167, February 2020.

[11] N. Dragoni, S. Giallorenzo, A. Lluch-Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," 06 2016.

[12] D. K. Rensin, *Kubernetes - Scheduling the Future at Cloud Scale*, 1005 Gravenstein Highway North Sebastopol, CA 95472, 2015. [Online]. Available: http://www.oreilly.com/webops-perf/free/kubernetes.csp

[13] "Aws wavelength," last accessed 25 November 2020. [Online]. Available: https://aws.amazon.com/wavelength/

[14] "iperf - the ultimate speed test tool for tcp, udp and sctp," last accessed 25 November 2020. [Online]. Available: https://iperf.fr/iperf-doc.php

[15] "Amazon ec2 on-demand pricing," last accessed 25 November 2020. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

[16] M. Fowler and J. Lewis, "Microservices," 2014, last accessed 25 November 2020. [Online]. Available: https://martinfowler.com/articles/microservices.html

[17] C. MacKenzie, K. Laskey, F. Mccabe, P. Brown, and R. Metz, "Reference model for service oriented architecture 1.0," *Public Rev. Draft*, vol. 2, 08 2006.

[18] B. Butzin, F. Golatowski, and D. Timmermann, "Microservices approach for the internet of things," 09 2016, pp. 1–6.

[19] T. Leppänen, C. Savaglio, L. Lovén, T. Järvenpää, R. Ehsani, E. Peltonen, G. Fortino, and J. Riekki, "Edge-based microservices architecture for internet of things: Mobility analysis case study," 12 2019.

[20] D. Sabella, V. Sukhomlinov, L. Trang, S. Kekki, P. Paglierani, R. Rossbach, X. Li, Y. Fang, D. Druta, F. Giust, L. Cominardi, W. Featherstone, B. Pike, S. Hadad, L. Sony, V. Fang, and B. Acs, "Developing software for multi-access edge computing," 02 2019.

[21] D. Adib, "The edge computing latency promise," last accessed 25 November 2020. [Online]. Available: https://stlpartners.com/edge-computing/edge-computing-architecture-impact-latency/

[22] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An overview on edge computing research," *IEEE Access*, vol. PP, pp. 1–1, 01 2020.

[23] H. Bangui, S. Rakrak, S. Raghay, and B. Buhnova, "Moving to the edge-cloud-of-things: Recent advances and future research directions," vol. 7, p. 309, 11 2018.

[24] J. Wang, J. Pan, F. Esposito, P. Calyam, Z. Yang, and P. Mohapatra, "Edge cloud offloading algorithms: Issues, methods, and perspectives," 06 2018.

[25] P. Varshney and Y. Simmhan, "Characterizing application scheduling on edge, fog, and cloud computing resources," *Software: Practice and Experience*, vol. 50, no. 5, p. 558–595, May 2020. [Online]. Available: http://dx.doi.org/10.1002/spe.2699

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.

[26] T. Elgamal, "Costless: Optimizing cost of serverless computing through function fusion and placement," in *2018 IEEE/ACM Symposium on Edge Computing (SEC)*, 2018, pp. 300–312.
[27] J. Ren, G. Yu, Y. He, and G. Li, "Collaborative cloud and edge computing for latency minimization," *IEEE Transactions on Vehicular Technology*, vol. PP, pp. 1–1, 03 2019.
[28] A. Das, S. Imai, S. Patterson, and M. P. Wittie, "Performance optimization for edge-cloud serverless platforms via dynamic task placement," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, 2020, pp. 41–50.
[29] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl, "Maui: Making smartphones last longer with code offload," vol. 2010, 10 2010, pp. 49–62.
[30] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "Clonecloud: Elastic execution between mobile device and cloud," 01 2011, pp. 301–314.
[31] H. Wu, W. Knottenbelt, and K. Wolter, "An efficient application partitioning algorithm in mobile environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, pp. 1464 – 1480, 01 2019.
[32] K. Li, "A game theoretic approach to computation offloading strategy optimization for non-cooperative users in mobile edge computing," *IEEE Transactions on Sustainable Computing*, pp. 1–1, 2018.
[33] B. Yang, Z. Li, and W. Liu, "Non-cooperative game approach for task offloading in edge clouds," 2018.
[34] S. Ranadheera, S. Maghsudi, and E. Hossain, "Mobile edge computation offloading using game theory and reinforcement learning," 2017.
[35] H. M. Fard, R. Prodan, J. J. D. Barrionuevo, and T. Fahringer, "A multi-objective approach for workflow scheduling in heterogeneous environments," in *2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (ccgrid 2012)*, 2012, pp. 300–309.
[36] N. Hassan, K.-L. Yau, and C. Wu, "Edge computing in 5g: A review," *IEEE Access*, vol. PP, pp. 1–1, 08 2019.
[37] "Transformational performance with 5g and edge computing," last accessed 25 November 2020. [Online]. Available: https://d1.awsstatic.com/Wavelength2020/AWS-5G-edge-Infographic-FINAL-Aug2020-2.pdf
[38] M. Maheswaran, R. Wenger, R. Olaniyan, S. Memon, O. Fadahunsi, and R. Echomgbe, "A language for programming edge clouds for next generation iot applications," 06 2019.
[39] S. Maheshwari, D. Raychaudhuri, I. Seskar, and F. Bronzino, "Scalability and performance evaluation of edge cloud systems for latency constrained applications," 10 2018.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:46:51 UTC from IEEE Xplore. Restrictions apply.